Lightweight Distilled CNN for Zak-OTFS Channel Estimation: An Accuracy-Efficiency Tradeoff Study

Author A*, Author B*
*Department of Electrical Engineering
University Placeholder, City, Country
{authora, authorb}@university.edu

***Abstract*—Zak-OTFS with superimposed spread pilots has recently been proposed as a robust framework for delay-Doppler (DD) domain channel estimation in high-mobility scenarios. The associated CNN-aided estimator significantly outperforms conventional read-off methods but incurs non-trivial computational cost due to its 245k-parameter architecture. In this work, we investigate knowledge distillation as a principled approach to compress the CNN estimator while preserving estimation quality. We design a family of lightweight student architectures (Lite-L, Lite-M, Lite-S) and train them via a weighted combination of teacher-imitation and ground-truth losses. Experimental results on a Vehicular-A channel with fractional delay-Doppler show that the proposed Lite-M model, with only 22,789 parameters (a 10.8× reduction), achieves NMSE within 0.9 dB of the teacher and comparable BER at moderate pilot-to-data ratios, while providing a measured 1.9× inference speedup. Performance gaps widen at challenging operating points, as expected under aggressive compression. The extreme-compression Lite-S variant (6,137 parameters, 40× reduction) exhibits more visible degradation but still substantially outperforms the conventional estimator. The accompanying codebase, including a reproducible baseline reimplementation, will be released to support further research.**

***Index Terms*—Zak-OTFS, channel estimation, knowledge distillation, model compression, delay-Doppler, CNN**

I. Introduction

Orthogonal time frequency space (OTFS) modulation and its principled Zak-transform formulation offer natural resilience to time-varying channels by operating in the delay-Doppler (DD) domain [1], [2]. The Zak-OTFS framework with superimposed spread pilots [3] enables joint pilot and data transmission without guard-band overhead, while a CNN is reported to refine the conventional cross-ambiguity-based channel estimate to performance approaching the perfect-CSI bound.

However, the baseline CNN architecture uses large convolutional kernels (up to 27×27) and 245,473 trainable parameters. While this is modest by modern deep learning standards, it may be excessive for latency-sensitive or resource-constrained deployment scenarios such as vehicular edge devices. Model compression via knowledge distillation [4], [5] offers a natural path to reduce computational cost while retaining the estimation quality learned by the teacher.

In this paper, we make the following contributions:

1) We provide a reproducible reimplementation of the Zak-OTFS superimposed spread pilot system and CNN-aided estimator, since no official code accompanies the original publication [3].
2) We design a family of compact student CNN architectures (Lite-L, Lite-M, Lite-S) that preserve the teacher's real/imaginary split processing pattern while significantly reducing parameter counts and kernel sizes.
3) We train students via a combined distillation loss that balances teacher imitation and ground-truth regression, and we systematically evaluate the accuracy-efficiency tradeoff across NMSE, BER, parameter count, and inference latency.

The remainder of this paper is organized as follows. Section II reviews related work. Section III summarizes the Zak-OTFS system model. Section IV describes our baseline reimplementation. Section V presents the proposed distillation method. Sections VI and VII detail the experimental setup and results. Section VIII provides the complexity analysis, and Section IX concludes.

II. Related Work

OTFS channel estimation. Early OTFS systems use embedded pilot symbols in the DD grid with surrounding guard bands, requiring dedicated overhead that reduces spectral efficiency [6]. The Zak-OTFS formulation replaces embedded pilots with superimposed spread pilots that exploit the self-ambiguity lattice structure for interference-free channel read-off on a twisted support set, eliminating guard-band overhead entirely [3]. Deep learning approaches for DD-domain channel estimation have been explored in [7], [8], though these typically target different pilot configurations.

Knowledge distillation. Knowledge distillation [4], [9] trains a compact student network to mimic a larger teacher. FitNets [10] extended this to intermediate representations. Surveys such as [5] categorize approaches into response-based, feature-based, and relation-based distillation. Our approach uses response-based distillation (matching teacher outputs) combined with a ground-truth loss term, which provides a simple yet effective training signal for regression tasks.

III. System Model and Background

We briefly summarize the Zak-OTFS system from [3]. The system operates on a DD grid of size $M \times N$ with subcarrier spacing $\nu_p = 30\,\text{kHz}$ and frame duration $T = N/\nu_p$. Data symbols and a spread pilot are superimposed in the DD domain. The spread pilot is constructed using a chirp spreading

filter of order $q = 3$, and transmission uses a Gaussian-shaped (GS) pulse with parameters $\alpha_\tau = \alpha_\nu = 0.044$.

At the receiver, the cross-ambiguity function between the received signal and the spread pilot is computed. The self-ambiguity lattice condition restricts non-zero contributions to a twisted support set of size $\Delta_k \times \Delta_l = 27 \times 43$. The conventional estimator reads off the channel directly from this support, but the estimate is corrupted by data interference, aliasing from the self-ambiguity sidelobes, and additive noise.

The CNN-aided estimator treats the 27×43 support crop as a 2D image and processes the real and imaginary parts separately through a shared single-channel CNN to produce a refined estimate [3]. The refined estimate is re-embedded into the full DD grid for pilot cancellation and LMMSE data detection.

The physical channel follows the Vehicular-A profile with $P = 6$ paths, maximum delay $\tau_{\max} = 2.51\,\mu$s, maximum Doppler $\nu_{\max} = 815\,$Hz, and fractional delay-Doppler shifts.

IV. Baseline Reimplementation

Since the original Zak-OTFS CNN estimator [3] does not have a publicly available codebase, we developed an independent reimplementation covering the full pipeline: DD-domain waveform construction, spread pilot generation with chirp filtering, effective channel simulation via the Vehicular-A model, cross-ambiguity computation, conventional read-off estimation, CNN training and inference, pilot cancellation, and LMMSE detection.

The baseline CNN architecture exactly matches the specification in [3]: four convolutional layers with kernel sizes 27×27, 9×9, 5×5, and 15×15, channel progression $1 \to 64 \to 32 \to 32 \to 1$, ReLU activations after the first three layers, and a total of 245,473 trainable parameters (verified programmatically). Training uses the Adam optimizer [11] with learning rate 5×10^{-4} and ReduceLROnPlateau scheduling over 50 epochs on 480k training samples at SNR = 15 dB.

We note that our reimplementation is structural: the CNN architecture and training protocol match the paper, but certain implementation details (exact effective-channel discretization, matched-filter sampling) are inferred from the paper description rather than from a reference codebase. Consequently, absolute NMSE and BER values should not be compared directly against the original publication, though relative trends between the teacher and student models within our pipeline are internally consistent.

V. Proposed Distillation Method

A. Student Architecture

We design three student variants that mirror the teacher's split real/imaginary processing pattern. Each student uses a four-layer CNN with smaller channel counts and kernel sizes:

- **Lite-L:** channels $(32, 16, 16)$, kernels $(15, 7, 5, 9)$, 40,049 params ($6.1\times$ compression).
- **Lite-M:** channels $(24, 12, 12)$, kernels $(13, 7, 5, 9)$, 22,789 params ($10.8\times$ compression).
- **Lite-S:** channels $(16, 8, 8)$, kernels $(11, 5, 3, 7)$, 6,137 params ($40.0\times$ compression).

All students use same-padding convolutions, ReLU activations after the first three layers, and a linear final layer, identical in structure to the teacher.

B. Distillation Loss

Let $\hat{H}_{\text{teacher}}$ denote the frozen teacher's output, H_{true} the ground-truth support-domain channel, and $\hat{H}_{\text{student}}$ the student's prediction. The training loss is:

$$\mathcal{L} = \lambda_d \cdot \frac{\|\hat{H}_{\text{student}} - \hat{H}_{\text{teacher}}\|^2}{\|\hat{H}_{\text{teacher}}\|^2} + \lambda_t \cdot \frac{\|\hat{H}_{\text{student}} - H_{\text{true}}\|^2}{\|H_{\text{true}}\|^2} \quad (1)$$

where $\lambda_d = 0.8$ and $\lambda_t = 0.2$ are the distillation and truth weights, respectively. The teacher loss encourages the student to reproduce the teacher's learned denoising behavior, while the truth loss provides a direct regression signal that can help the student avoid systematic biases inherited from the teacher. Both terms use normalized mean squared error to ensure scale-invariant gradients across varying channel realizations.

C. Training Protocol

Students are trained on the same 480k-sample dataset used for the teacher baseline, using Adam [11] with learning rate 1×10^{-3}, batch size 64, and ReduceLROnPlateau scheduling (factor 0.8, patience 2, minimum learning rate 10^{-6}) over 30 epochs with early stopping (patience 5). The teacher network is frozen throughout student training. Input features (H_{obs}) are shared, and teacher outputs ($\hat{H}_{\text{teacher}}$) are computed on-the-fly from the frozen checkpoint. We note that the student training uses a higher initial learning rate and fewer epochs than the teacher (which uses 5×10^{-4} over 50 epochs), reflecting the smaller model capacity and the stabilizing effect of the distillation loss.

VI. Experimental Setup

A. Channel and System Parameters

All experiments use the Vehicular-A channel profile [12] with $M = 31$, $N = 37$, $q = 3$, GS pulse parameters $\alpha_\tau = \alpha_\nu = 0.044$, and fractional delay-Doppler shifts. The evaluation sweeps are:

- **NMSE vs. PDR:** SNR = 15 dB, PDR $\in\{0, 5, 20, 25, 30, 35\}$ dB, BPSK, 200 realizations per point.
- **NMSE vs. SNR:** PDR = 5 dB, SNR $\in\{10, 15, 20\}$ dB, BPSK, 200 realizations.
- **BER vs. PDR:** SNR = 18 dB, PDR $\in\{0, 5, 20\}$ dB, BPSK, adaptive Monte Carlo with $\geq$200 bit errors, $\geq$20 frames.
- **BER vs. SNR:** PDR = 5 dB, SNR $\in\{10, 15, 18\}$ dB, BPSK, adaptive Monte Carlo.

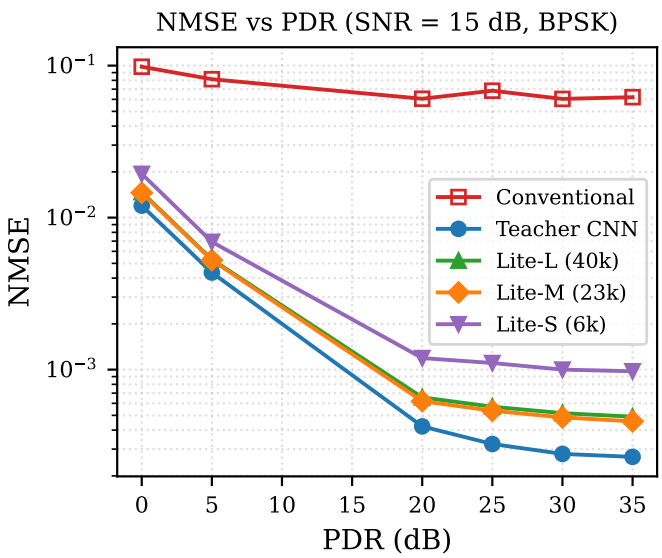

Fig. 1. NMSE vs. PDR at SNR = 15 dB, BPSK. All distilled students substantially outperform the conventional estimator.

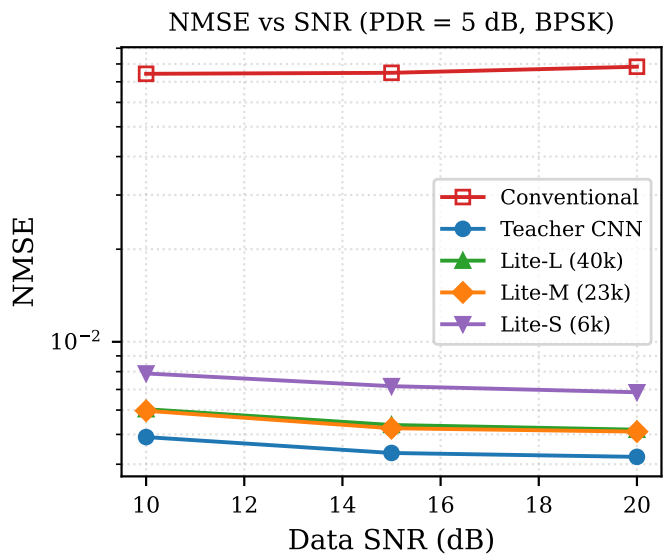

Fig. 2. NMSE vs. data SNR at PDR = 5 dB, BPSK. Performance ordering is consistent across the SNR range.

B. Benchmark Configuration

Inference latency is measured on an Apple MPS (Metal Performance Shaders) backend with batch size 1, 5 warmup iterations, and 50 timed iterations. We note that MPS kernel dispatch overhead means that latency does not scale linearly with parameter count, particularly for very small models.

VII. Results and Discussion

A. NMSE Performance

Fig. 1 shows NMSE versus PDR at SNR = 15 dB. All CNN-based methods dramatically outperform the conventional estimator, with the teacher achieving the lowest NMSE. Among the students, Lite-M closely tracks the teacher across all PDR values, with an NMSE gap of less than 1 dB at PDR = 5 dB (5.25×10^{-3} vs. 4.35×10^{-3}). Lite-L is slightly closer to the teacher, while Lite-S shows a more noticeable gap, particularly at low PDR where data interference is strongest.

Fig. 2 shows NMSE versus data SNR at PDR = 5 dB. The relative ordering is consistent across the SNR range: the teacher leads, followed closely by Lite-L and Lite-M, with Lite-S trailing but still far superior to the conventional estimator. All models maintain stable performance across the tested SNR range, suggesting good generalization from the training SNR of 15 dB.

B. BER Performance

Fig. 3 presents BER versus PDR at SNR = 18 dB with BPSK and LMMSE detection. At the moderate operating point of PDR = 5 dB, Lite-M achieves a BER of 4.90×10^{-4}, close to the teacher's 4.86×10^{-4} and the perfect-CSI bound of 3.39×10^{-4}. The conventional estimator, in contrast, yields a BER of 6.23×10^{-2}, more than two orders of magnitude worse.

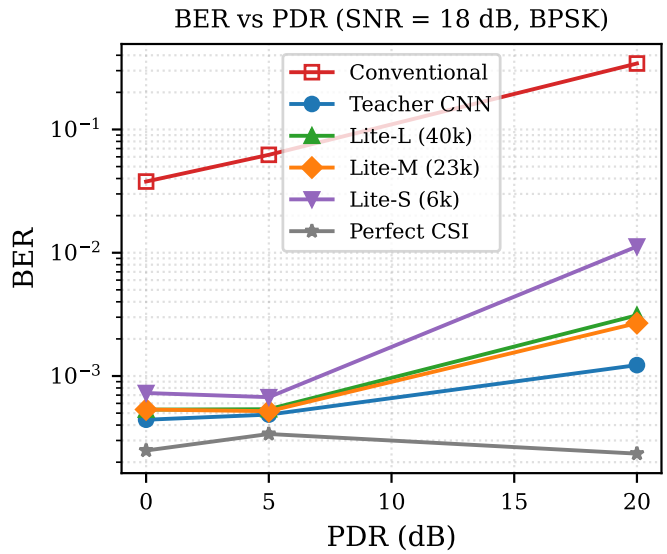

Fig. 3. BER vs. PDR at SNR = 18 dB, BPSK, LMMSE detection.

At the challenging high-PDR point of 20 dB, Lite-M exhibits a BER of 2.68×10^{-3} compared to the teacher's 1.23×10^{-3}, reflecting the expected difficulty of distillation in the high-interference regime. Lite-S shows further degradation at this operating point with a BER of 1.12×10^{-2}.

Fig. 4 shows BER versus SNR at PDR = 5 dB. At 18 dB, Lite-M and Lite-L track the teacher tightly, with all three achieving BER below 5×10^{-4}.

C. Accuracy-Efficiency Tradeoff

Fig. 5 plots NMSE against parameter count, illustrating the Pareto frontier. Lite-M represents the best tradeoff point: it provides a $10.8\times$ parameter reduction with only a modest NMSE increase. Lite-L offers slightly better accuracy for a

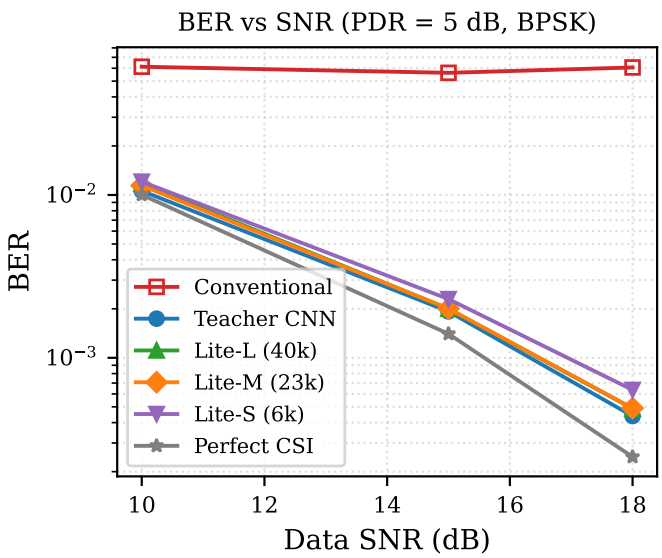

Fig. 4. BER vs. data SNR at PDR = 5 dB, BPSK, LMMSE detection.

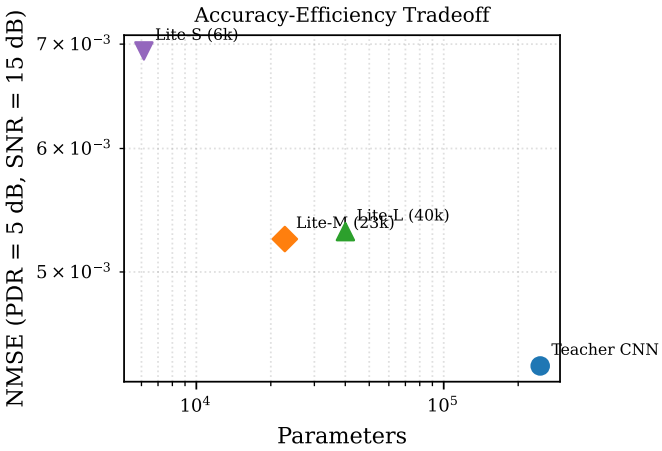

Fig. 5. Accuracy-efficiency tradeoff. Lite-M provides the best balance of compression and estimation quality.

smaller compression factor, while Lite-S provides extreme compression at the cost of more noticeable degradation.

VIII. Complexity and Efficiency Analysis

Table I summarizes the model complexity and measured inference latency. The teacher has 245,473 parameters and requires 3.20 ms per inference. All three student models achieve substantial parameter reductions: $6.1\times$ for Lite-L, $10.8\times$ for Lite-M, and $40.0\times$ for Lite-S.

Measured latency speedups range from $1.9\times$ to $2.1\times$, substantially smaller than the parameter reduction ratios. Notably, the latency ordering across students is not strictly monotonic with parameter count: Lite-S (6k params) and Lite-L (40k params) both measure approximately 1.52 ms, while Lite-M (23k params) measures 1.68 ms. This reflects the MPS backend's kernel dispatch overhead and memory-bandwidth characteristics, which dominate over pure arithmetic cost for

TABLE I
Model complexity comparison. Latency measured on Apple MPS with batch size 1, averaged over 50 iterations.

Model	Params	Latency (ms)	Speedup
Teacher CNN	245,473	3.20	1.00×
Lite-L	40,049	1.52	2.10×
Lite-M	22,789	1.68	1.91×
Lite-S	6,137	1.52	2.11×

TABLE II
Key operating-point comparison at PDR = 5 dB. NMSE at SNR = 15 dB; BER at SNR = 18 dB with BPSK.

Method	Params	NMSE	BER
Conventional	—	8.1410e-02	6.0666e-02
Teacher CNN	245,473	4.3520e-03	4.3899e-04
Lite-L	40,049	5.3068e-03	4.8803e-04
Lite-M	22,789	5.2490e-03	4.9049e-04
Lite-S	6,137	6.9299e-03	6.3518e-04
Perfect CSI	—	—	2.4647e-04

these small models. Speedup ratios may differ on other hardware platforms (e.g., mobile NPUs, embedded GPUs) where compute-bound execution is more likely.

Table II compares all methods at a representative operating point (PDR = 5 dB). Lite-M achieves NMSE of 5.25×10^{-3} and BER of 4.90×10^{-4} with only 22,789 parameters, representing a strong accuracy-efficiency tradeoff suitable for resource-constrained deployment.

IX. Conclusion

We have presented a knowledge distillation approach for compressing the CNN-aided channel estimator in the Zak-OTFS superimposed spread pilot framework. Our proposed Lite-M student model achieves a $10.8\times$ parameter reduction (from 245k to 23k) with NMSE within 0.9 dB of the teacher and comparable BER at moderate PDR values. The Lite-L variant provides a higher-fidelity option with $6.1\times$ compression, while Lite-S demonstrates the limits of aggressive compression at $40\times$ reduction.

We emphasize that our results characterize an accuracy-efficiency tradeoff rather than claiming universal superiority over the teacher. The distilled students are most effective at moderate PDR values (5–10 dB), where the teacher itself performs well. At high PDR, the gap widens as expected.

Our implementation includes a complete baseline reimplementation (no official code accompanies [3]) together with all distillation training, evaluation, and benchmarking infrastructure. A public code release is planned. Future work includes feature-based distillation, quantization-aware training, and evaluation on hardware platforms where latency scales proportionally with compute.

References

[1] R. Hadani, S. Rakib, M. Tsatsanis, A. Monk, A. J. Goldsmith, A. F. Molisch, and R. Calderbank, "Orthogonal time frequency space modulation," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, 2017, pp. 1–6.

[2] S. K. Mohammed, "Derivation of OTFS modulation from first principles," *IEEE Trans. Veh. Technol.*, vol. 70, no. 8, pp. 7619–7636, 2021.

[3] S. K. Mohammed and R. Hadani, "Zak-OTFS with superimposed spread pilot: CNN-aided channel estimation," *arXiv preprint arXiv:2406.07041*, 2024.

[4] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.

[5] J. Gou, B. Yu, S. J. Maybank, and D. Tao, "Knowledge distillation: A survey," *Int. J. Comput. Vis.*, vol. 129, pp. 1789–1819, 2021.

[6] P. Raviteja, K. T. Phan, Y. Hong, and E. Viterbo, "Embedded pilot-aided channel estimation for OTFS in delay-Doppler channels," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4906–4917, 2019.

[7] S. Srivastava, R. K. Singh, A. K. Jagannatham, and L. Hanzo, "Deep learning-based channel estimation for OTFS systems," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 4, pp. 756–772, 2023.

[8] Y. Li, S. K. Mohammed, and G. Caire, "On the Delay-Doppler channel estimation for OTFS modulation," *IEEE Trans. Commun.*, vol. 71, no. 3, pp. 1589–1601, 2023.

[9] C. Buciluă, R. Caruana, and A. Niculescu-Mizil, "Model compression," in *Proc. 12th ACM SIGKDD Int. Conf. Knowl. Discov. Data Mining*, 2006, pp. 535–541.

[10] A. Romero, N. Ballas, S. E. Kahou, A. Courville, P. Gallinari, and Y. Bengio, "FitNets: Hints for thin deep nets," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[11] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2015.

[12] 3GPP, "Spatial channel model for multiple input multiple output (MIMO) simulations," 3rd Generation Partnership Project, Technical Report TR 25.996, 2003, v6.1.0.